Unified Engineering Workbench

PASS Science Works — R&D Internship 2026-2027, Common Technical Assignment

Name	Ajay Kumar Reddy Krishnareddy Gari
Roll No / Reg No	RA2311003020057
Programme	CSE Core A
Branch	SRMIST-RMP
Repository	github.com/ajaykumarreddy-k/RA2311003020057
Submission date	01 November 2026
Actual effort	Not recorded
Stack	C++17 core and HTTP server, plain HTML/JS front end, CMake, doctest

Every figure in this document was produced by building the pushed repository and running it on 1 November 2026 (Ubuntu 24.04, g++ 13.3.0, 1 vCPU). Things that were not run or not measured are listed under Known limitations.

1. Design and architecture

The workbench has four layers. Each depends only on the one above it in this list.

Layer	Path	Contents
Core library	`src/core/`	FEA solver, backpropagation, thread pool and benchmark, STL and ANSYS CSV parsers, embedded device mocks. No networking, no UI, no OS calls.
HTTP server	`src/server/`	REST endpoints, static file serving, async job manager (cpp-httplib, nlohmann/json, vendored in `third_party/`).
Launcher	`src/launcher/`	Finds a free localhost port, starts the server without a console window, opens the default browser.
Presentation	`web/`	Single page with four tabs (Geometry, Thermal Analysis, Performance, Hardware). Three.js and OrbitControls are vendored; no CDN.

The browser only draws results. Parsing, numerics and threading happen in C++. Long jobs (the benchmark) run on a worker thread and the page polls `/api/benchmark/status/:id`, so the UI stays responsive. Closing the tab calls `/api/shutdown` and the server exits.

Checked in this run: `GET /api/health` returned HTTP 200 with `{"status":"ok"}`, `GET /` returned 200 (36,882 bytes), and `POST /api/shutdown` stopped the server cleanly.

2. Thermal FEA (Task C)

Problem: `-d/dx(kA dT/dx) = qA` on a uniform rod with L = 1 m, A = 1e-4 m^2, k = 10 W/(m K), q = 10,000 W/m^3, T(0) = 100 C, T(L) = 20 C, insulated sides, constant properties. Exact solution: T(x) = 100 - 80x/L + [q/(2k)] x (L - x).

Formulation. N two-node linear elements of length h = L/N. Element stiffness `Ke = (kA/h)[[1,-1],[-1,1]]`, element load `Fe = (qAh/2)[1,1]`. The global system is tridiagonal. Both Dirichlet values are imposed by moving the known-node contributions to the right-hand side and solving for the N-1 interior nodes with the Thomas algorithm (`src/core/fea.cpp`).

Verification. Error is measured at the nodes and at element midpoints using the FE linear interpolation (the average of the two end-node values), against the exact solution.

N	h (m)	Max node error (C)	Max midpoint error (C)	Observed order
10	0.1000	2.84e-14	1.250000	-
20	0.0500	1.99e-13	0.312500	2.000
40	0.0250	8.81e-13	0.078125	2.000
80	0.0125	1.22e-12	0.019531	2.000

The nodal error stays near rounding level because the exact solution is quadratic and the Galerkin solution for this 1D problem is nodally exact. The midpoint error equals (q/(8k)) h^2 = 125 h^2, which gives 1.25, 0.3125, 0.078125 and 0.019531 for the four meshes, matching the table. Halving h divides the error by 4, so the order is 2. Raw data: `results/fea_nodes.csv` columns `N,x_m,T_fea_C,T_exact_C` (units in the column names).

Assumptions and debugging note. Constant k and q, steady state, Dirichlet ends only. Reporting only nodal error would have hidden the convergence behaviour, since nodal error is flat at 1e-13 to 1e-12. Moving to midpoint error with FE interpolation exposed the $O(h^2)$ trend.

3. Backpropagation (Task F)

Model: h = tanh(wx + b), y_hat = v h + c, J = 0.5 $(y_hat - y)^2$, with x = 1, y = 2, w = 0.5, b = 0, v = 1, c = 0, learning rate 0.1. Implemented in `src/core/backprop.cpp` with no automatic differentiation.

Forward pass: z = 0.50000000, h = 0.46211716, y_hat = 0.46211716, J = 1.18254182.

Parameter	Analytic dJ/d(param)	Central difference (eps = 1e-6)	Abs. error	Pass (≤ 1e-5)
w	-1.20946448	-1.20946448	1.09e-10	yes
b	-1.20946448	-1.20946448	1.09e-10	yes
v	-0.71068205	-0.71068205	5.25e-11	yes
c	-1.53788284	-1.53788284	1.12e-10	yes

Chain rule: dJ/dc = e, dJ/dv = e h, dJ/db = e v $(1 - h^2)$, dJ/dw = e v $(1 - h^2)$ x, where e = y_hat - y = -1.53788284. All four gradients come from the original parameters, then one simultaneous update gives w = 0.62095, b = 0.12095, v = 1.07107, c = 0.15379. Each parameter moves against its gradient, so the loss falls. Over the 100 further steps the loss goes from 1.18254182 to 1.703e-20 (101 recorded values, `results/backprop_loss.csv`). With one sample and four parameters the network can fit the target exactly, so the loss tends to zero.

4. Multicore execution (Task D)

A seeded batch generator (`std::mt19937`, seed 42, k in [1, 50], q in [1000, 50000]) produces independent cases. The serial solver and a bounded `std::thread` pool solve identical cases with the same method, and results are stored in input order. Timing uses `std::chrono::steady_clock` around the solves only, with one warm-up and five measured repetitions, and reports the median.

Item	Value measured in this run
CPU	Intel Xeon @ 2.10 GHz, 1 logical CPU
RAM	3,995 MB
Compiler and flags	g++ 13.3.0, `-std=c++17 -O3 -pthread`
Workload	500 cases, 100 elements each
1 worker, median of 5	2.294 ms (speedup 1.00, efficiency 1.00 by definition)
2 and 4 workers vs serial	max |T_serial - T_parallel| = 0.0 C (limit 1e-8)

> This machine has one logical CPU, so the benchmark caps workers at 1 and speedup cannot be assessed here. The brief allows this case. The parallel path was still run with 2 and 4 workers to confirm it matches the serial results. No speedup or efficiency figures for 2 or 4 workers are claimed. The Performance tab measures them on any multi-core machine.

Expected limits on a multi-core machine: each case is a small tridiagonal solve (about 4.6 microseconds here), so queue locking and thread start-up cost take a large share of the time, and efficiency will fall below 100 percent as workers increase.

5. CAD and ANSYS data (Task B)

Both parsers are C++ (`stl_parser.cpp`, `ansys_parser.cpp`). The STL parser reads ASCII STL, checks the token structure, rejects NaN and infinite values, and reports line-level errors. The CSV parser requires the header `node_id,x_m,y_m,z_m,temperature_C`, validates integer IDs and finite numbers, and rejects empty data with an explanatory message. Both return bounding boxes, and the CSV result includes the temperature range.

The front end draws STL triangles with rotate, pan, zoom, fit-to-view and a wireframe toggle. CSV nodes are drawn as a coloured 3D point cloud with a temperature legend in C and click-to-inspect (node ID, x, y, z in m, temperature in C).

Example file	Content	Units and provenance
`data/synthetic_heat_sink.stl`	Generated heat-sink-like bracket, 12 KB	Self-created, synthetic; units in `PROVENANCE.md`
`data/synthetic_ansys_thermal.csv`	160 nodes, 5.9 KB	Self-created and labelled synthetic; it is not an actual ANSYS export. Coordinates m, temperature C.

Size limits documented in `data/PROVENANCE.md`: STL tested to about 10 MB, CSV to about 5 MB. Interactive picking slows above roughly 50,000 points (CPU raycasting). Binary STL is not supported. The browser rendering itself was not exercised in this environment (no browser session); the parsers and the API were.

6. Embedded integration design (Task E)

Design and mocks are in `docs/embedded_design.md` and `src/core/embedded_sim.cpp`. The host connects to three devices: a microcontroller board (STM32H743-class, USB CDC serial), an FPGA board (Artix-7-class, SPI) and a JTAG adapter (FTDI FT2232H-class, USB). The document gives the block diagram, the interface and voltage-domain table (power, ground, level shifting, connectors, host link, JTAG signal ownership) and a safe bring-up plan.

The JTAG adapter is a debug and programming interface. It shifts bits through a boundary-scan chain under the host's control; it has no application bus, flow control or compute resources, so it cannot stand in for the microcontroller or the FPGA board. The three boards also differ in connector, voltage and protocol, so none is assumed interchangeable.

The mock adapters provide discovery, capability reporting, connect and disconnect, one telemetry message per device, and an injected timeout followed by a retry that returns the device to the connected state. All of it is simulated and labelled SIMULATED in the Hardware tab. No physical hardware was used.

7. Test outcomes

Built with CMake (Release) and run with CTest on 1 November 2026: 5 of 5 suites passed in 0.02 s.

Suite	Cases	Assertions	Covers
test_fea	5	185	Boundary values, midpoint error trend, order 2, N = 1, invalid L, A, k, N
test_backprop	4	132	Forward pass, gradients, finite-difference check, simultaneous update, 100 steps
test_multicore	4	271	Pool, seeded batch, serial vs parallel agreement, timing harness
test_parsers	4	37	Valid files, empty files, missing tokens, NaN or Inf, bad columns
test_embedded	3	41	Discovery, connect, telemetry, fault and recovery
Total	**20**	**666**	

8. Known limitations, AI use and ownership

- **No Windows executable was built or tested.** Development and every run above were on Linux. `Launch.exe` comes from `build_release.bat` (MSVC, static runtime), which has not been run. The clean-machine launch on Windows 11 is unverified.
- Multi-core speedup was not measured (single-CPU environment, see section 4).
- The web page was checked through HTTP requests only; no screenshots or browser run are included, and there is no demo video in the repository.
- The Task F derivation sheets in Appendix A are AI-generated, not the candidate's own handwriting.

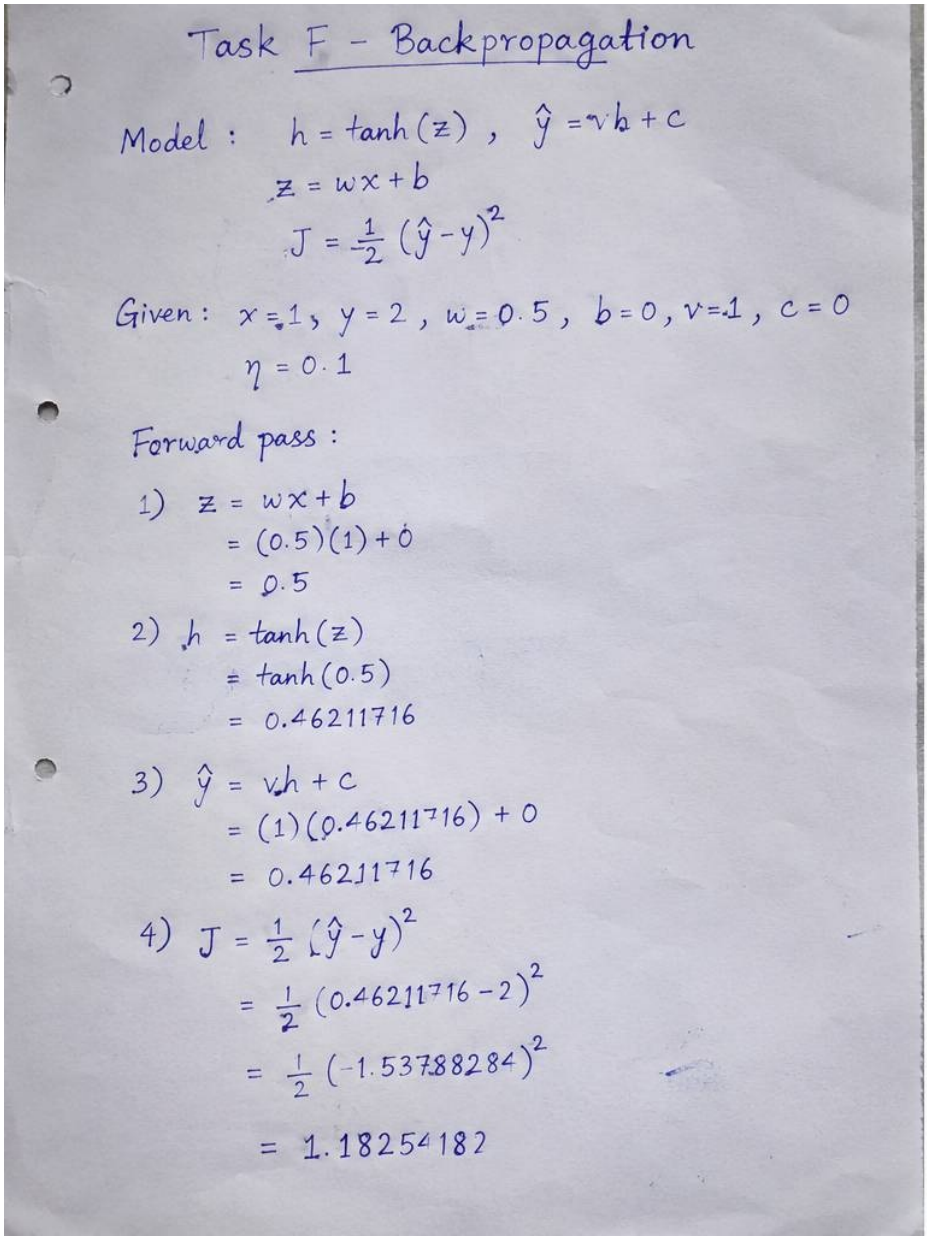
- ASCII STL only; picking limit above about 50,000 points; steady-state, Dirichlet-only 1D FEA; single-sample backprop; embedded devices are simulated.
- The pushed Git history contains one commit for this project (see Appendix B).

AI use. The code was written with an AI coding assistant, recorded in `AI_USE_LOG.md` as Gemini via the Antigravity environment. The log documents two incorrect outputs that were corrected: a test file missing `DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN` (linker errors), and CTest tests registered without a working directory (relative paths to `data/` failed). A later review of this report found and corrected earlier draft figures that did not match a fresh run (gradient digits, node errors, test assertion count, and unsupported multi-core timings); the figures here are from that fresh run.

Ownership. The application code, tests and documents were produced with AI assistance and reviewed by the candidate. Reused third-party code is limited to the vendored libraries: cpp-httplib, nlohmann/json, doctest, Three.js and OrbitControls, all MIT licensed (versions in `OWNERSHIP.md`).

Appendix A. Backpropagation derivation sheets (Task F)

These two sheets were **generated with AI assistance and are not the candidate's own handwriting**. They are included as a disclosed AI-produced derivation. The values were checked against the C++ run in section 3 (forward pass, three gradients, update). The brief's requirement for the candidate's own handwritten pages is therefore not met by this appendix.



Task F - Backpropagation

Model : $h = \tanh(z)$, $\hat{y} = vh + c$

$z = wx + b$

$J = \frac{1}{2}(\hat{y} - y)^2$

Given : $x = 1$, $y = 2$, $w = 0.5$, $b = 0$, $v = 1$, $c = 0$

$\eta = 0.1$

Forward pass :

1) $z = wx + b$
$= (0.5)(1) + 0$
$= 0.5$

2) $h = \tanh(z)$
$= \tanh(0.5)$
$= 0.46211716$

3) $\hat{y} = vh + c$
$= (1)(0.46211716) + 0$
$= 0.46211716$

4) $J = \frac{1}{2}(\hat{y} - y)^2$
$= \frac{1}{2}(0.46211716 - 2)^2$
$= \frac{1}{2}(-1.53788284)^2$
$= 1.18254182$

Chain rule derivatives :

1) $\frac{dJ}{dw} = (\hat{y} - y)\, v\, (1 - h^2)\, x$
$= (-1.53788284)(1)(1 - 0.46211716^2)(1)$
$= -1.20946448$

2) $\frac{dJ}{db} = (\hat{y} - y)\, v\, (1 - h^2)$
$= (-1.53788284)(1)(1 - 0.46211716^2)$
$= -1.20946448$

3) $\frac{dJ}{dv} = (\hat{y} - y)\, h$
$= (-1.53788284)(0.46211716)$
$= -0.71068205$

4) $\frac{dJ}{dc} = (\hat{y} - y)$
$= -1.53788284$

Gradient descent update (simultaneous, $\eta = 0.1$) :

$$\begin{cases} w' = w - \eta \frac{dJ}{dw} = 0.5 - 0.1(-1.20946448) = 0.620946448 \\ b' = b - \eta \frac{dJ}{db} = 0 - 0.1(-1.20946448) = 0.120946448 \\ v' = v - \eta \frac{dJ}{dv} = 1 - 0.1(-0.71068205) = 1.071068205 \\ c' = c - \eta \frac{dJ}{dc} = 0 - 0.1(-1.53788284) = 0.153788284 \end{cases}$$

Note : The gradient points in the direction of steepest increase of J. By subtracting $\eta \nabla J$, we move in the opposite direction (downhill), which decreases the loss J.

Appendix B. Git history and AI-use summary

Pushed repository history (`git log` on `main`, cloned 1 November 2026):

```
a6a7219 2026-11-01 Add Unified Engineering Workbench project
2007b8d 2026-05-02 Update README.md
6bf63d5 2026-05-02 Readme Fix
bbb88ce 2026-05-02 Final - Update with output
74fca38 2026-05-02 Final Push all implemented ... front end to the middle ware to default api
8405b48 2026-05-02 Pre-Commit Before Final push - 401 error Auth Failed for api ...
cfd163f 2026-05-02 Refining the Design and Applied Custom Google themed style ...
3cc80b9 2026-05-02 Removed the DeadCode increased code quality ...
a28e6eb 2026-05-02 Set Up Final Packagers and the Main Components and react hooks Basic
817e1aa 2026-05-02 Adding Basic Material theme Inspired by the Google
440fc9c 2026-05-02 first commit - Basic Template of Material Ui
```

Commit a6a7219 adds the whole workbench and removes the files of an earlier React and Material UI project that occupied the same repository. The ten older commits belong to that earlier project and are not part of this assignment. The workbench's module-by-module development history is not in the pushed repository.

AI-use summary (full prompt matrix in `AI_USE_LOG.md`):

Area	Prompt purpose	Candidate check
FEA	1D linear elements, Dirichlet BCs, midpoint convergence	Nodal exactness explained; midpoint error matched 125 h^2; order 2
Backprop	Gradients, central-difference check, 100 steps	Simultaneous-update semantics checked; errors about 1e-10
Multicore	Thread pool, seeded batch, timing harness	Timing limited to solves; single-CPU case handled
Parsers	Strict STL and CSV validation	CTest working-directory bug fixed
Embedded	Mock MCU, FPGA, JTAG with timeout recovery	Enum typo fixed; JTAG rationale written
Server and UI	REST server, offline Three.js page, clean shutdown	Health, index and shutdown checked over HTTP